

AIM- To create and execute two FreeRTOS tasks with different priorities and analyzing their effect on LED blinking behaviour.

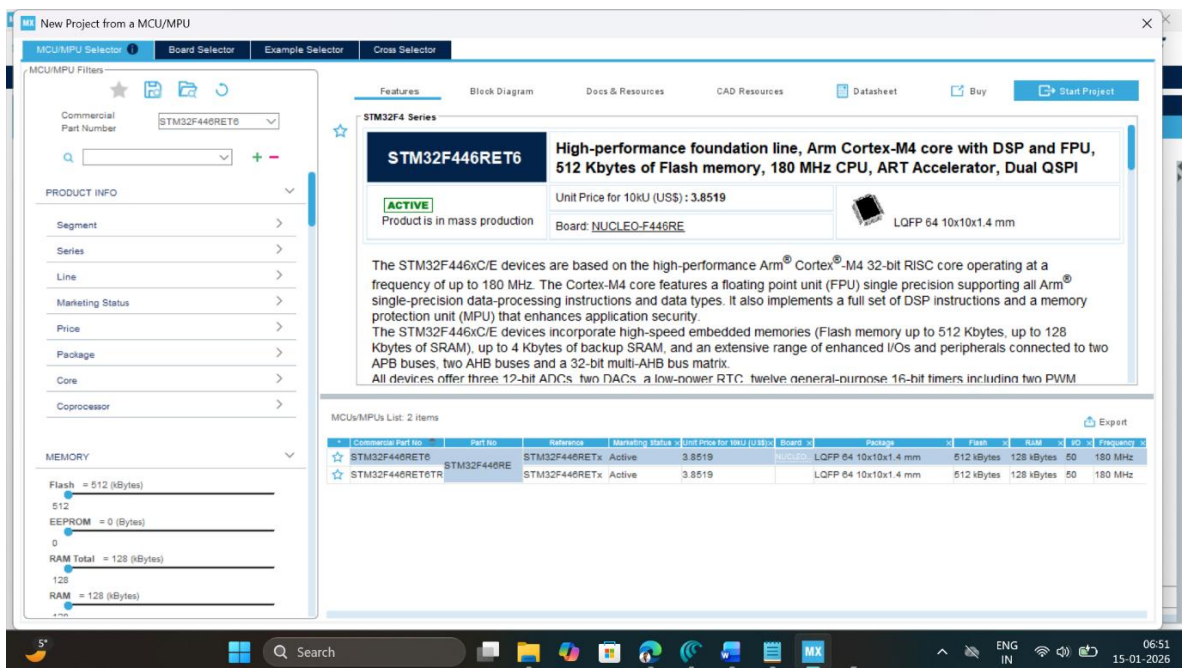
APPARATUS- STM32 Nucleo-F446RE development board, an LED, Breadboard, Connecting wire, Resistance, USB Type-A to Mini-B cable, STM32 CUBE IDE, STM32 CubeMX.

THEORY

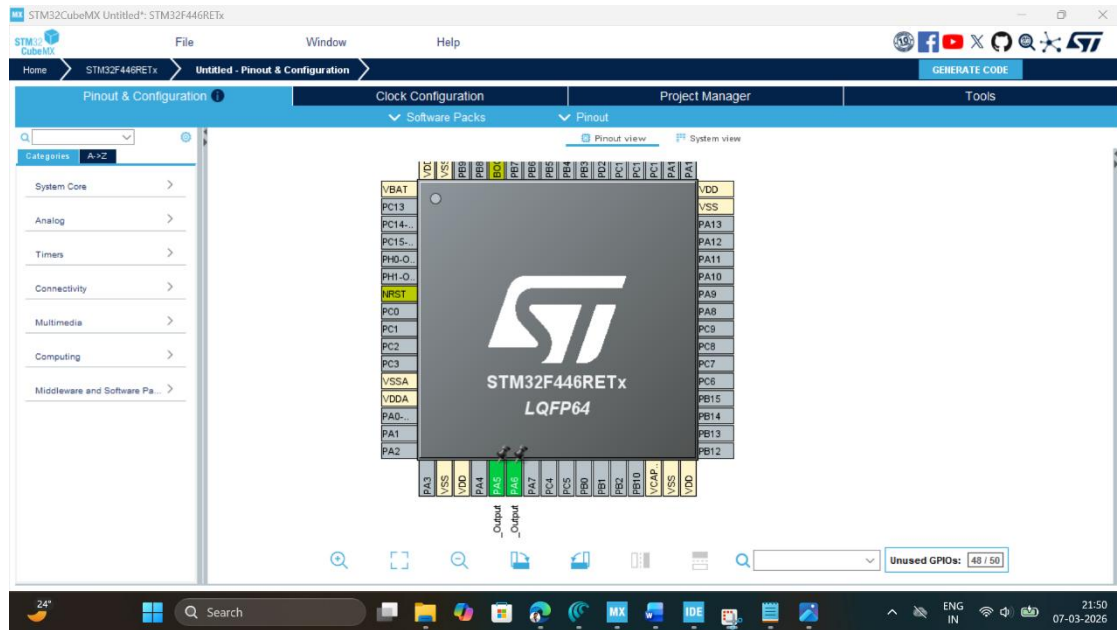
FreeRTOS implements preemptive priority-based scheduling where tasks with higher priority values (e.g., `osPriorityHigh` > `osPriorityNormal` > `osPriorityLow`) execute immediately when they become Ready, preempting lower-priority tasks even mid-execution. In this experiment, two tasks with identical `osDelay(500)` periods are assigned different priority levels via CMSIS-RTOS v2 task attributes, demonstrating how the scheduler favours higher-priority tasks for CPU allocation. When both tasks block cooperatively via `osDelay()`, lower-priority tasks still execute periodically; however, as priority differences increase, the lower-priority task experiences progressive CPU starvation, observable through reduced LED blinking rates and decreased message frequency in the SWV ITM console. This illustrates the fundamental trade-off between priority responsiveness and fairness in real-time systems.

PROCEDURE

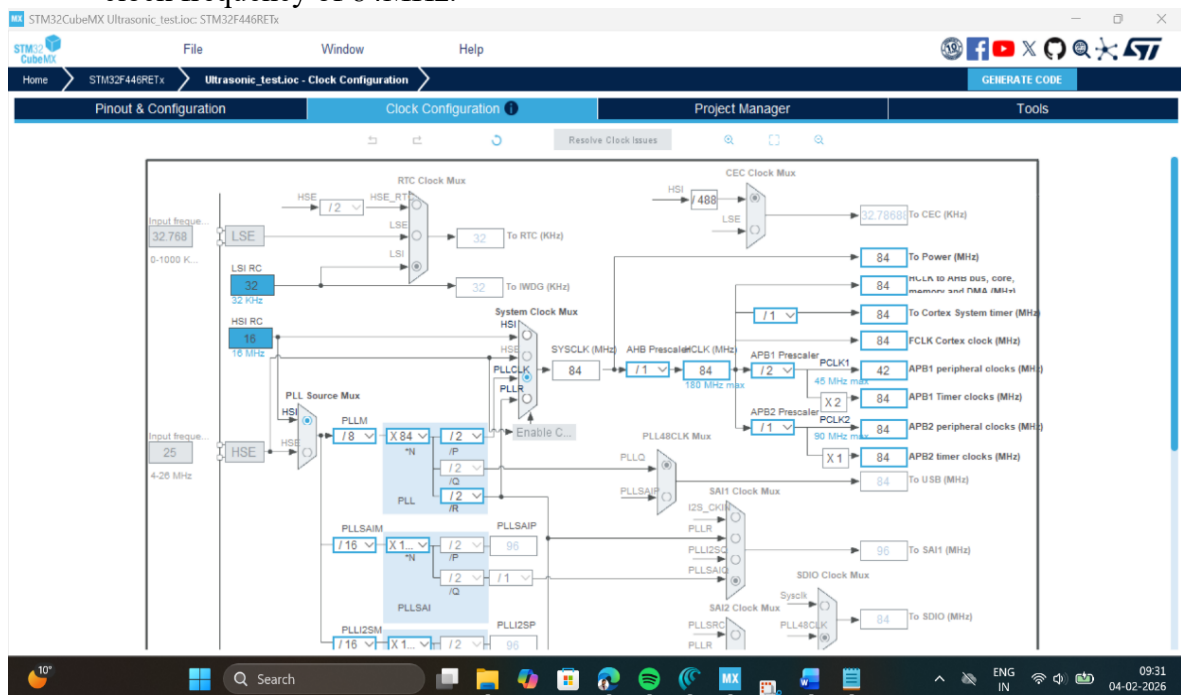
1. Open CubeMX and Click on ACCESS TO MCU SELECTOR.



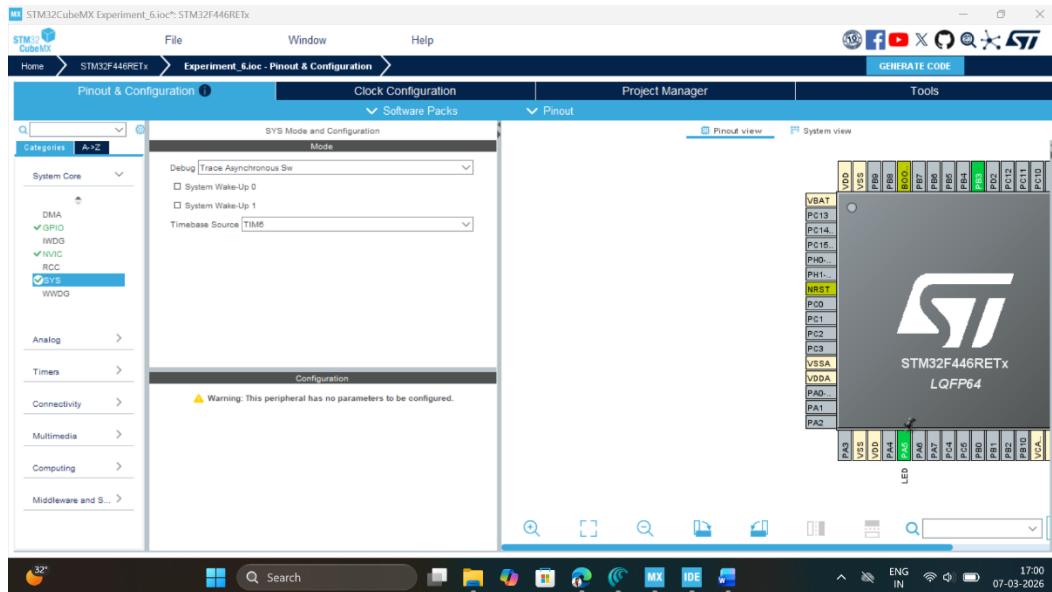
2. Write and select the microcontroller as you want to use and then click start the project. Select commercial part number STM32L446RE.
3. In system Core select GPIO and right click on pin PA5 and PA6 to make it as GPIO output pin.



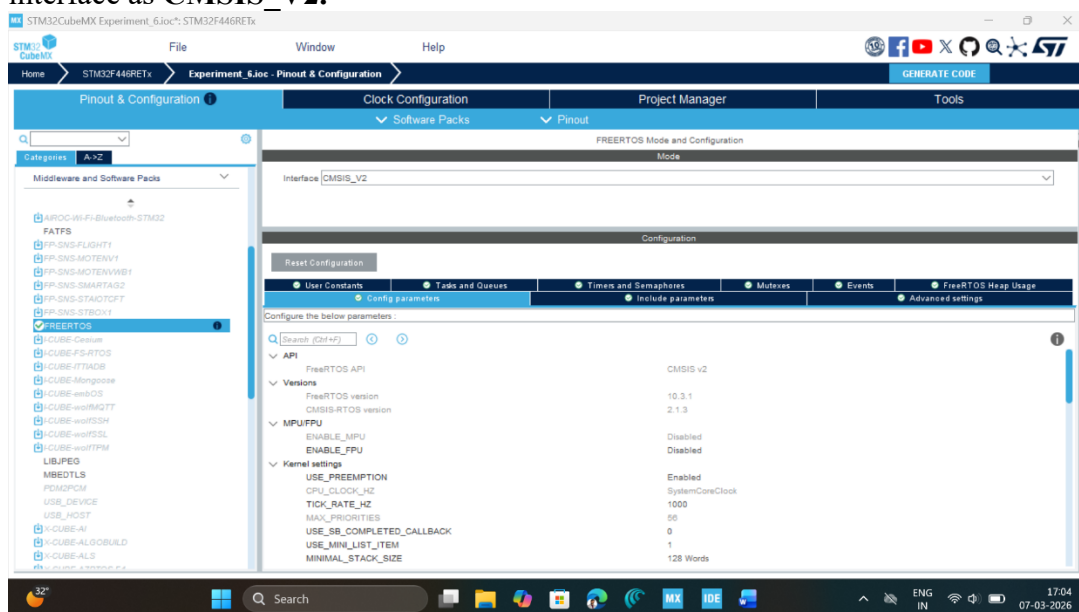
4. **Clock Configuration:** Configure clock as shown below to get maximum internal clock frequency of 84MHz.



5. **SYS Mode:** Click on SYS mode and select debug as *Trace Asynchronous Sw*, and Timebase source to **TIM6**.



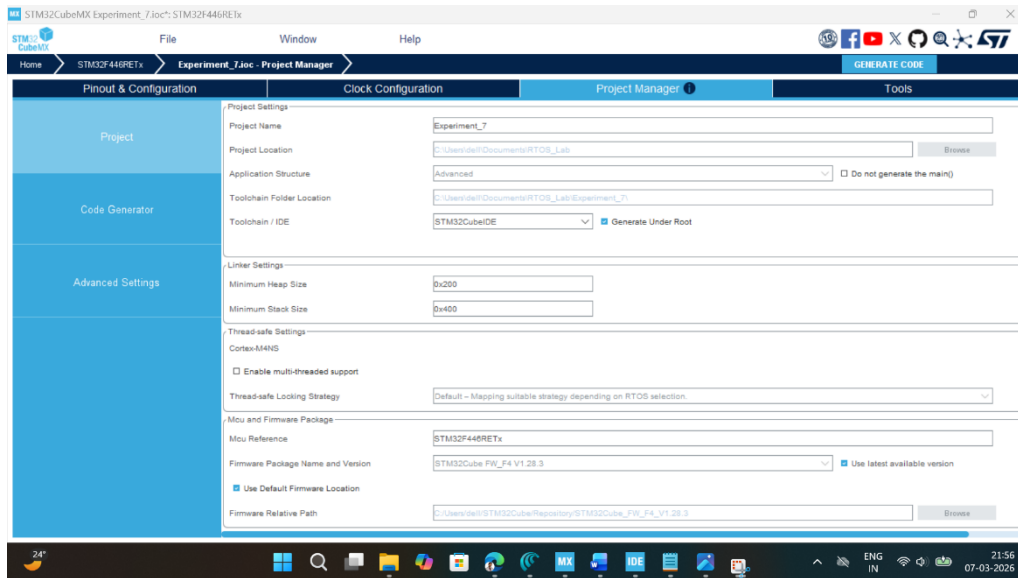
6. **FreeRTOS Configuration:** Click on Middleware and Software Pack, then select FreeRTOS software pack, which is already available with STMCube Mx. Select interface as CMSIS_V2.



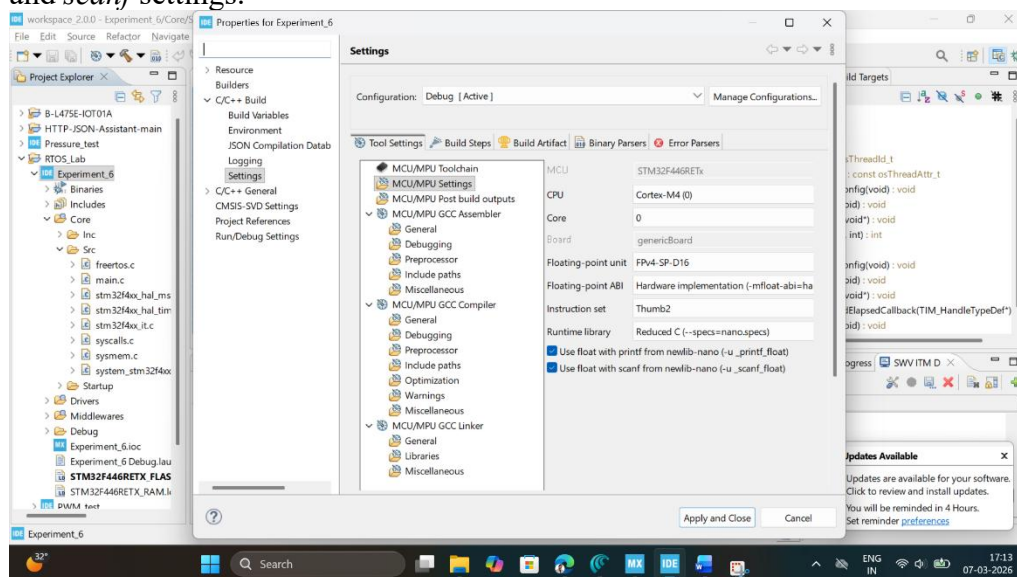
7. Click on Task & Queue Tab of configuration setting and create two tasks as LED_1 and LED_2 as shown below. Keep priority of both tasks same.

Tasks									
Task Name	Priority	Stack Size	Entry Function	Code Generation Opt.	Parameter	Allocation	Buffer Name	Control Block Name	
LED_1	osPriorityNormal	128	StartDefaultTask	Default	NULL	Dynamic	NULL	NULL	
LED_2	osPriorityNormal	128	StartLED_2	Default	NULL	Dynamic	NULL	NULL	

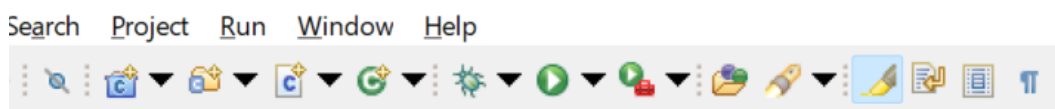
8. Write Project name and select IDE.



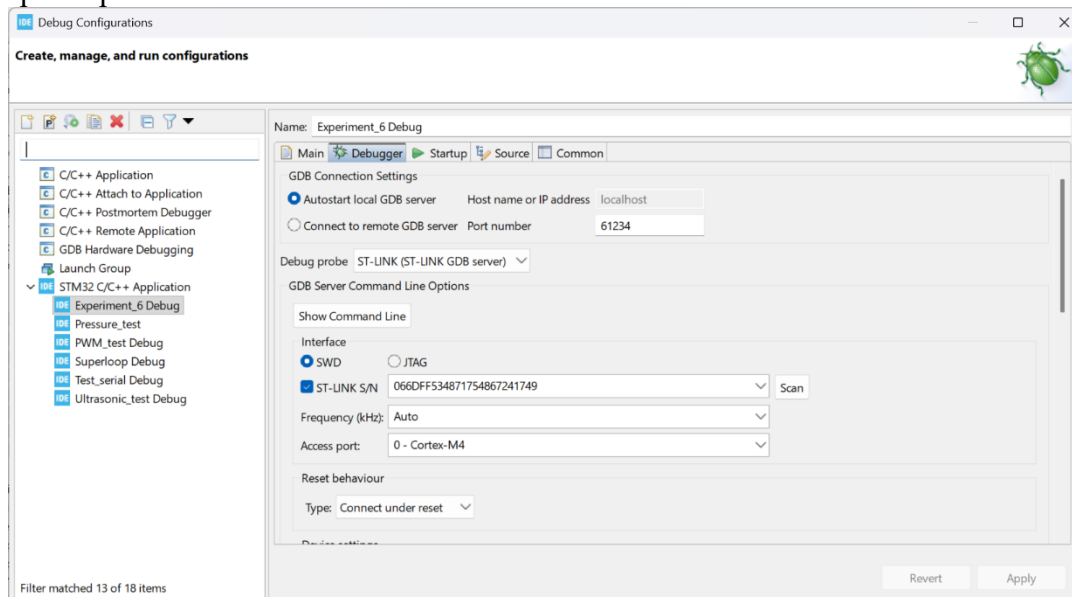
9. Click on generate code.
10. Click on open project.
11. Click 'OK' on the generated popup showing "successfully imported the project on the workspace" and the project will be successfully added to the cube IDE.
12. Open the main source code on Cube IDE by clicking on **main.c**
13. Click on project properties, go to C/C++ Build → settings → check box for *printf* and *scanf* settings.



14. Now click on Debug icon and configure debugger for SWV ITM Trace
- 15.



16. Click on Debugger configuration, select debugger console, and click on ST-LINK S/N and then scan, as we attach a STM32F446RE development board, it will automatically pick up ST-LINK id.



17. Next, select the Serial Wire Viewer (SWV) and check box on enable button and make sure to pass on the right core clock frequency which is set-up at per point 4 here i.e. 84MHz. Then click on Apply button and close this window.
18. After configuration write following instructions in the **main.c** file and compile the program and ensure there will be zero error after compiling.

```
/* USER CODE BEGIN Includes */
#include <stdio.h>
/* USER CODE END Includes */
```

Write the function to enable SWV ITM Console to trace the output of Task_1

```
/* USER CODE BEGIN 0 */
// Retarget fwrite to ITM
int _write(int file, char *ptr, int len) {
    for (int i = 0; i < len; i++) {
        ITM_SendChar(*ptr++);
    }
    return len;
}
/* USER CODE END 0 */
```

Edit the Task_1 and Task_2 function as given below to toggle the inbuilt LED for 500ms delay.

```
* USER CODE END Header Task1_function */
void Task1_function(void *argument)
{
    /* USER CODE BEGIN 5 */
    /* Infinite loop */
    for(;;)
    {
```

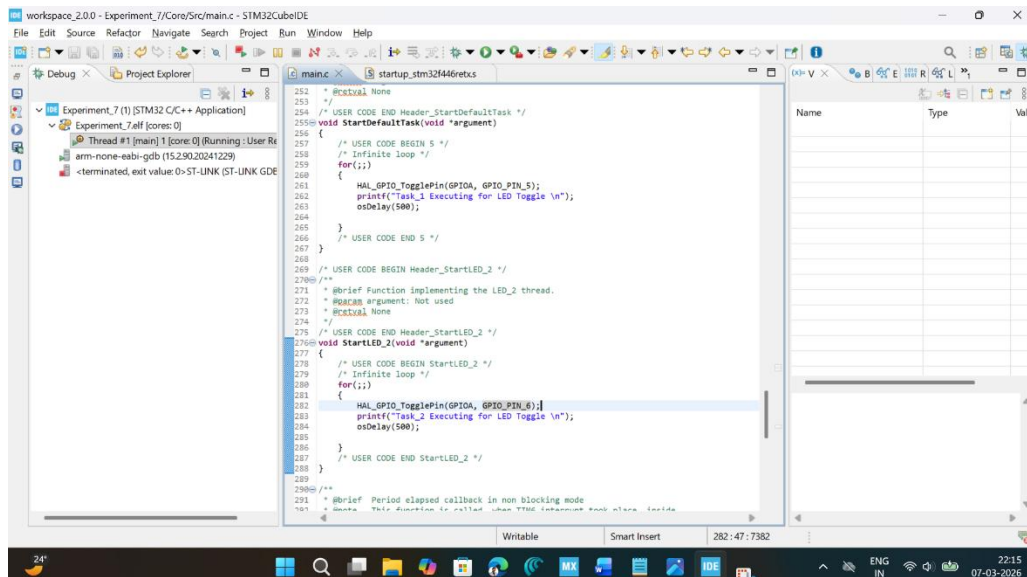
```

HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
printf("Task_1 Executing for LED Toggle \n");
osDelay(500);
}
}

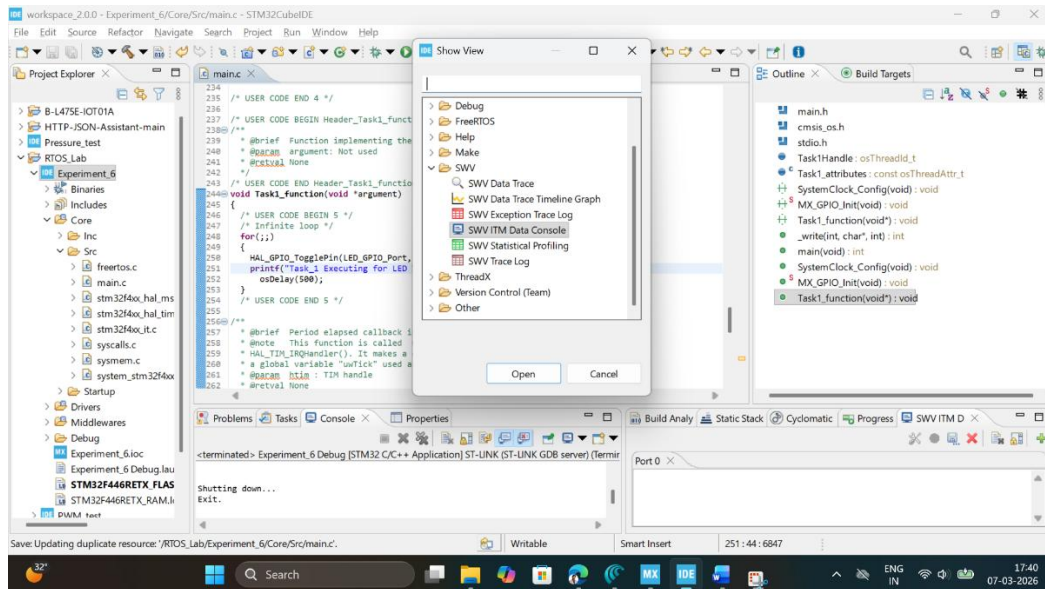
void StartLED_2(void *argument)
{
    /* USER CODE BEGIN StartLED_2 */
    /* Infinite loop */
    for(;;)
    {
        HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_6);
        printf("Task_2 Executing for LED Toggle \n");
        osDelay(500);
    }
    /* USER CODE END StartLED_2 */
}

/* USER CODE END 5 */

```

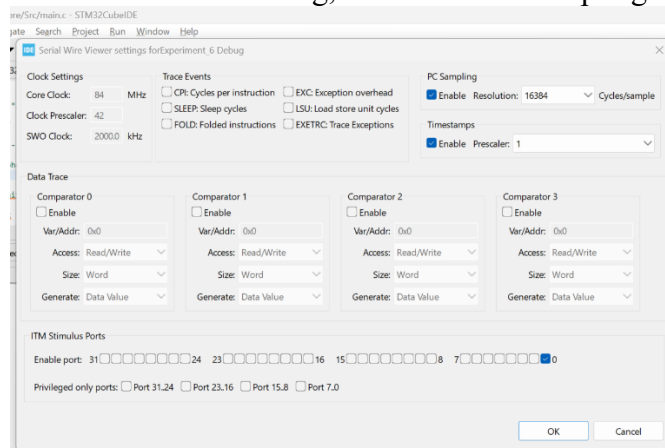


19. Now, click on Window tab, select show view → other → select SWV ITM Data Console.



20. Now click on Debug button, start debugging this main.c file and click on switch to change the perspective to debugger.

21. Click on SWV ITM Data Console setting, enable the PC sampling and select port 0.



22. Click on Start Trace (Red Dot) on SWV ITM Data Console tab and then click on resume button to start executing the main program. Now you can see the message on Port 0 and LED Toggle on STM32F446RE development board.

The screenshot shows the STM32CubeIDE workspace. The main.c file is open, displaying code for two tasks, Task_1 and Task_2, which are configured to toggle LEDs. The SWV ITM Data Console is open, showing the output of the tasks. The console output shows alternating messages for Task_1 and Task_2, indicating that the tasks are being scheduled and executed alternately. The status bar at the bottom indicates the current line is 97:1:2754.

OBSERVATION:

S. No	Task_1 Priority	Task_2 Priority	LED_1 Blink rate (Hz)	LED_2 Blink rate (Hz)	SWV Console message	Remarks
1	osPriorityLow	osPriorityLow	~1 Hz	~1 Hz	Both messages appear alternately	Equal sharing of CPU, both tasks fairly
2	osPriorityNormal	osPriorityLow	~1 Hz	~1 Hz (slightly delayed)	Task_1 messages more frequent	Task_1 gets more CPU preference
3	osPriorityHigh	osPriorityNormal	~1 Hz	~0.8–1 Hz	Task_1 dominates console output	Task_2 runs less frequently
4	osPriorityRealtime	osPriorityNormal	~1 Hz (very stable)	~0.5–0.8 Hz	Task_1 almost continuous	Task_2 faces partial starvation
5	osPriorityError	osPriorityLow	No execution	~1 Hz	Only Task_2 message visible	Invalid priority - Task_1 not scheduled

RESULT

The two tasks were created using FreeRTOS Middleware software pack using CMSIS V2 interface and the output of two tasks is observed using SWV ITM Data console Port 0 and analyzed for different task priorities.

Reflection Summary

- How does the scheduler share CPU time between two tasks of equal priority that both call `osDelay(500)`, and what evidence do you see of this in the LED blinking pattern and SWV ITM output?

① equal priority scheduling (OS Delay based)

- Both tasks call `osDelay(500)`
- So both go to Blocked state
- When delay ~~continuous~~ completes - both become Ready

→ Scheduler behaviour

- uses round robin scheduling
- Each task gets equal CPU time

2. If one of the tasks had no `osDelay()` or blocking call, how would this affect the execution of the other task, and what would you expect to observe on the LEDs and in the console messages?

2 → Important concept

- That task will never block
- Always stays in Running / Ready state

→ Results

- It will continuously occupy CPU
- Other task will get almost no chance

3. Compare this two-task RTOS implementation with a single super-loop that toggles two LEDs: what advantages in structure, readability, and extensibility do you observe when using separate tasks?

3

```
while(1);
{
  toggle LED1;
  HAL-Delay(500);
  toggle LED2;
  HAL-Delay(500);
}
```

problems

- Sequential execution
- Not Scalable

→ RTOS

```
Task1 → LED1
Task2 → LED2
```

Advantage.

- parallel behaviour
- Better structure
- Real time response

4. How does SWV ITM tracing support the verification of correct RTOS behaviour (task execution order, timing, liveness) without adding extra GPIO pins or UART peripherals?

4

Role of SWV ITM Tracing

→ what it shows

- Task execution message
- Timing behaviour
- frequency of execution

→ why powerful

- No extra hardware needed
- No UART wiring